

Architecture

Cassandra uses so many components to build upon its complex architectural theory that it is really difficult to go through all the bits and pieces without missing anything. The terminologies discussed here are those that provide an insight into the inner workings of this database. Cassandra's architecture is built more towards avoiding a single point of failure in the cluster, so as to have unhindered access to the maximum amount of data in case any part of the cluster fails. It uses technologies that resemble peer-to-peer networking to achieve a failure-proof data distribution model. Hence, no single node in a Cassandra cluster can be termed as a master of others, and coordination among the nodes is achieved with the help of the Gossip failure detection protocol, which is used to determine the availability of a node in the cluster. Gossip is managed with the help of the Gossiper present on each node, which keeps on initiating 'Gossip' communications periodically with random nodes to check their availability. As a result, each node performs similar functions to others, and there are no designated roles for a particular function.

Each node in Cassandra is part of a ring, which determines the way in which the topology of a Cassandra cluster is represented. Each node in the cluster is assigned a token, and a part of the data for which it is responsible. The data to be assigned to each node is determined by the partitioner, which allows the row keys to be sorted according to the partitioning strategy chosen. The default strategy is random partitioning, which works on the basis of consistent hashing to distribute row keys. Another partitioning strategy available is the use of Byte-Ordered Partitioner, which orders row keys according to their raw bytes. AntiEntropy is then used to synchronise the replicas of the data to the newest version by periodically comparing the checksums. Merkle trees are used in Cassandra to implement AntiEntropy, just like for Dynamo, but in a slightly different way. For more details, you could read the respective documentation.

Reads and writes

When a node receives a read request from the client, it first determines the consistency level specified in it, on the basis of which it determines the number of replicas that need to be contacted. When each replica responds with the requested data, it is then compared to determine the most recent version to be reported back. If the consistency level specified is a weaker one, then the latest revision is immediately reported back, and then out-of-date replicas are updated. If it is one of the stronger consistency levels, then first a read repair operation is performed, after which the data is reported back.

In case of a write operation, the consistency level is again used to determine the amount of nodes required to respond with an acknowledgement of success, before the whole operation is deemed to be successful. If the nodes required for consistency are not available, then mechanisms like

'hinted handoff' are used to ensure consistency whenever the node comes back online. The complete flow for a write operation involves components like the commit logs, memtables, SSTables, etc. The commit logs are the first failover protection mechanism, where the operation is written so that the written data can be recovered in case of a failure. The memtables then act as an in-memory database, where all the data is updated until it is flushed to disk in the form of SSTables. Compaction is then periodically performed to assimilate data, so that it can be merged into a single file.

The data model

The data model for Cassandra is column-family-based, which resembles the design for relational databases using tables. In this case, however, the data model works in a fundamentally different manner to fixed-size columns. In Cassandra, the database schema is not required to be fixed before the application is used—and it can actually be updated on the fly without any problems on the server. In most cases, the client can arbitrarily determine the number of columns and metadata it wants to store in a particular column family. This results in much more flexibility for the application and in rows that vary in size and in the number of columns.

Keyspaces are used to define the namespace for each application's column families. Hence, these can be defined on a per-application basis, or according to any other schema design that is required. Column families can be of two types—*static* and *dynamic*. The static column family allows predefined columns in which a value may or may not be stored. Dynamic column families, on the other hand, allow the application to define columns whenever they are needed according to their usage. While specifying a column, its name, value and timestamp are needed. So basically, a column is the most basic unit in which a piece of information can be stored. Cassandra also supports structures like super columns and composite columns, which allow for further nesting.

Almost all RDBMS' require you to specify a datatype for each column. Though Cassandra does not have any such strong requirements, it allows specification of comparators and validators, which act like a datatype for the column name and the column's value, respectively. Except for counters, almost all datatypes like integer, float, double, text, Boolean, etc. can be used for comparators and validators.

Installation and configuration

The binary for installing Cassandra can be found at cassandra.apache.org/download. This is available in the form of a compressed file, which can be easily extracted on any OS. Before installation, ensure that Java 1.6 is available on the system, and that the `JAVA_HOME` environment variable has been set. Once the extracted files have been placed in the desired directory, the following commands can be used (on Linux) to set up the environment: